Tutorial on assessing benefits of using continuous outcomes to prioritise patient care

The purpose of this tutorial is to link the methods and results of the main manuscript with the code in the GitHub repository (https://github.com/robinblythe/Ranking_method_HEA). We will briefly go through the rationale for the study before setting up a simulation, using the `predictNMB` package, then apply our method to a machine learning model to predict 30 day mortality in emergency patients.

Background

This work was motivated by observing that many published prediction models for clinical deterioration, sepsis, and a variety of other clinical outcomes do not use predicted probabilities when the model is deployed. The underlying algorithm is capable of generating continuous values, but these are then categorised into risk groups, e.g., green-yellow-red, for the sake of convenience and reductions in cognitive burden. This is permissible when the risk of all patients within those groups is roughly equal. However, that is rarely the case; models will output a yellow or red alert, and then clinicians must reconstruct the patient's underlying risk based on clinical judgement. This is effectively duplicating the work done by the model.

Our proposed solution is to simply retain those continuous probabilities and consider using them for ranking patients within risk groups. The risk group system can still be used, but relative priorities (ranks based on predicted risks) within those risk groups should be informed by the model. If clinicians wish to increase or decrease a patient's relative priority in light of their risk group and ranking, that is just part of hospital medicine, but discarding the continuous probability entirely is wasteful.

Simulation setup

The data underlying this study include over 200,000 emergency department presentations at Singapore General Hospital. If you are interested in conducting research on emergency presentations, you may contact Duke-NUS Medical School and enquire about the ED2A dataset. It contains presentations from 2008 to 2020 and is a wonderful resource for informatics research. However, we don't actually need it yet. The first part of the study uses the `predictNMB` package to simulate predicted probabilities.

We encourage the use of the `renv` package for version control, as packages update over time. You can see what versions we're using in the lockfile. We'll be making use of a few packages:

```r
library(tidyverse) # Data wrangling and visualisation
library(furrr) # Parallel processing using futures
library(patchwork) # Stitching together publication-ready graphics
library(pROC) # Estimating area under the curve
library(predictNMB) # Generating samples and risk group thresholds
library(pmsampsize) # Estimating minimum sample sizes for simulated models
```

The conditions we want to test should mimic a range of prevalence (frequency of the predicted outcome) and AUC (area under the receiver operating characteristic curve - the discrimination of the model) values; ideally ones that are representative of hospital medicine. We also want to measure the effect of differences in model calibration, because many deployed models are often miscalibrated. We'll create a matrix of the combinations of these values we're interested in.

```r
combs <- expand_grid(
  event_rate = c(0.01, 0.05, 0.1),
  auc = c(0.65, 0.75, 0.85, 0.95),
  miscalibration = c("none", "underestimates", "overestimates")
)
```

Simulation: one iteration

Given there are 36 combinations to test, that might take awhile. To begin with, let's just look at the first row: a prevalence of 0.01, an AUC of 0.65, and no miscalibration. We can use the `furrr` package later to quickly run the different starting conditions using parallel processing.

We'll also set some broader simulation parameters. Let's assume we're working with a 1,000 bed hospital, but can only evaluate around 40 patients within recommended timeframes if they all have simultaneous alerts. We'll repeat the simulation process over 10,000 iterations - basically generating 10,000 sample datasets for the testing procedure.

```r
event_rate <- 0.01
auc <- 0.65
miscalibration <- "none"
n_test <- 1000
n_eval <- 0.04 * n_test
```

Minimum sample size for stable models

The `pmsampsize` package is a great resource and demonstrates sample size requirements for prediction models. We can use it to determine the conditions for training our hypothetical

prediction model. I typically specify slightly more parameters than predictors in the dataset to deal with potential interactions or non-linearity, so I'll specify 2 parameters here even if we're only using 1 in our hypothetical.

```
sampsize <- pmsampsize(
  type = "b",
  parameters = 2,
  prevalence = event_rate,
  cstatistic = auc
)
sampsize
```

```
Given input C-statistic = 0.65  & prevalence = 0.01
Cox-Snell R-sq = 0.0027

NB: Assuming 0.05 acceptable difference in apparent & adjusted R-squared
NB: Assuming 0.05 margin of error in estimation of intercept
NB: Events per Predictor Parameter (EPP) assumes prevalence = 0.01

           Samp_size Shrinkage Parameter CS_Rsq Max_Rsq Nag_Rsq   EPP
Criteria 1      6657     0.900         2 0.0027   0.106   0.025 33.29
Criteria 2       376     0.338         2 0.0027   0.106   0.025  1.88
Criteria 3        16     0.900         2 0.0027   0.106   0.025  0.08
Final           6657     0.900         2 0.0027   0.106   0.025 33.29

 Minimum sample size required for new model development based on user inputs = 6657,
 with 67 events (assuming an outcome prevalence = 0.01) and an EPP = 33.29
```

Given our inputs, we need a model with 6657 samples, and 67 minimum events. We can generate a dataset with these conditions from the `predictNMB` package, which uses Cohen's d to generate data where the AUC approaches the target AUC. We can then fit the trained hypothetical model and add the predicted probabilities to our training set. The get_sample() function is stochastic, so we want to set a seed for reproducibility.

```
set.seed(888)
train <- predictNMB::get_sample(
  auc = auc,
  n_samples = sampsize$sample_size,
  prevalence = event_rate,
  min_events = ceiling(sampsize$events)
```

```
)

fit <- glm(actual ~ x, data = train, family = binomial())
train$predicted <- predict(fit, type = "response")
```

Selecting thresholds for the model

One handy feature of `predictNMB` is that, given a model and some economic inputs, it can generate a range of thresholds (cutpoints) to create risk groups. The economic inputs are used for the Net Monetary Benefit (NMB) threshold, which weighs misclassification costs when it creates cutpoints. We'll also test two more popular cutpoints, the Youden index and one based on the false positive rate - the Number Needed to Evaluate (NNE). Some preliminary research has shown that for clinical deterioration, the NNE might be around 14.

Economic inputs

We will assume some very specific economic consequences of misclassification here. This is just for the NMB cutpoint - the other cutpoints don't use it. We'll assume that unobserved deterioration leads to ICU admission (cost = 14345) with 0.03 QALYs lost. We can avert this outcome with early detection, with an effectiveness of 1 minus the risk ratio of 0.910. The process of following up on early detection is the cost of staff time multiplied by the duration of the assessment, plus the opportunity cost - the time that we could better spend on other patients. The `get_nmb_sampler` function creates another function, which we'll call nmb(), which prints the confusion matrix (2 x 2 table).

```
wtp <- 45000
nmb <- get_nmb_sampler(
  # Cost of ICU admission
  outcome_cost = 14345,
  # Willingness to pay per QALY
  wtp = wtp,
  # QALYs lost due to deterioration event
  qalys_lost = 0.03,
  # Cost of an evaluation = (Clinician time cost * duration of MET) +
  #   (
  #  Opportunity cost = chance of successful intervention *
  #     outcome cost * underlying p0
  #   )
  high_risk_group_treatment_cost = (1.50 * 19) +
    ((1 - 0.910) * (14345 * 0.85) * (1.03)^4 * event_rate),
  # Chance of successful intervention
```

```
  high_risk_group_treatment_effect = 1 - 0.910
)
nmb()
```

```
                            TP                             FP
                   -14323.30125                    -40.85125
                            TN                             FN
                       0.00000                  -15695.00000
                    qalys_lost                            wtp
                       0.03000                   45000.00000
                   outcome_cost high_risk_group_treatment_effect
                    14345.00000                        0.09000
  high_risk_group_treatment_cost  low_risk_group_treatment_effect
                         40.85125                        0.00000
   low_risk_group_treatment_cost
                         0.00000
```

Based on our inputs, we can see that a false positive costs around $41. We can now get our risk thresholds using the `get_thresholds()` function. We'll also extract the Youden and NMB cutpoints.

```
thresholds <- get_thresholds(
  predicted = train$predicted,
  actual = train$actual,
  nmb = c(
    "TP" = nmb()[["TP"]],
    "TN" = nmb()[["TN"]],
    "FP" = nmb()[["FP"]],
    "FN" = nmb()[["FN"]]
  )
)
thresholds
```

```
             all             none value_optimising           youden
     0.000000000      1.000000000      0.025421126      0.011002866
 cost_minimising   prod_sens_spec            roc01   index_of_union
     0.028920215      0.010112700      0.010112700      0.009334028
```

```
cutpoint_nmb <- thresholds[["value_optimising"]]
cutpoint_youden <- thresholds[["youden"]]
```

NNE threshold

The NNE threshold needs a little bit of finesse because it's actually the inverse of the positive predictive value. We need to first generate the ROC curve, then extract the positive predictive value from that object, and take the inverse. The cutpoint is selected by taking the lowest threshold for which the NNE is 14.

```
nne <- 14

roc_curve <- roc(predictor = train$predicted, response = train$actual)
```

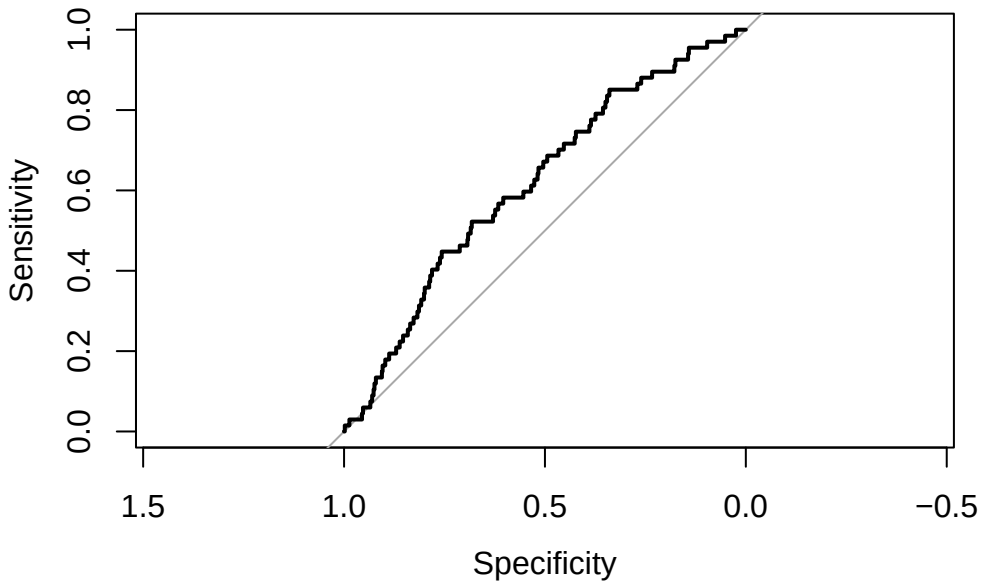
```
Setting levels: control = 0, case = 1
```

```
Setting direction: controls < cases
```

```
roc_curve
```

```

Call:
roc.default(response = train$actual, predictor = train$predicted)

Data: train$predicted in 7295 controls (train$actual 0) < 67 cases (train$actual 1).
Area under the curve: 0.6173
```

```
plot(roc_curve)
```




```
ppv <- coords(
  roc_curve,
  x = "all", input = "threshold", ret = c("threshold", "ppv")
)
ppv$nne <- 1 / ppv$ppv

cutpoint_nne <- min(ppv$threshold[ppv$nne == nne], na.rm = T)
```

Note that sometimes, just due to sampling variation and our starting conditions, the threshold will be infinitely low. In these cases, we just want to replace it with 0 (we want to evaluate every patient). We can create a helper function that just smooths out this whole process for us:

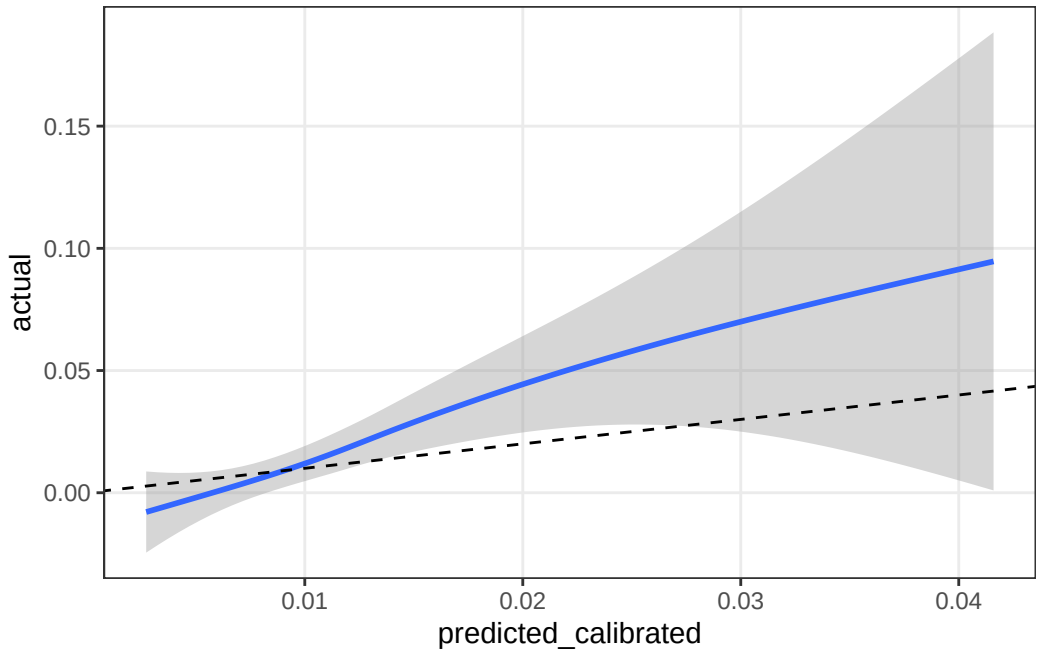
```
get_nne_threshold <- function(predictor, response, nne) {
  roc_curve <- roc(predictor = predictor, response = response)
  ppv <- coords(
    roc_curve,
    x = "all", input = "threshold", ret = c("threshold", "ppv")
  )
  ppv$nne <- 1 / ppv$ppv
  cutpoint <- min(ppv$threshold[ppv$nne == nne], na.rm = T) |> suppressWarnings()
```

```
  if_else(is.infinite(cutpoint), 0, cutpoint)
}
```

Generating the test sets

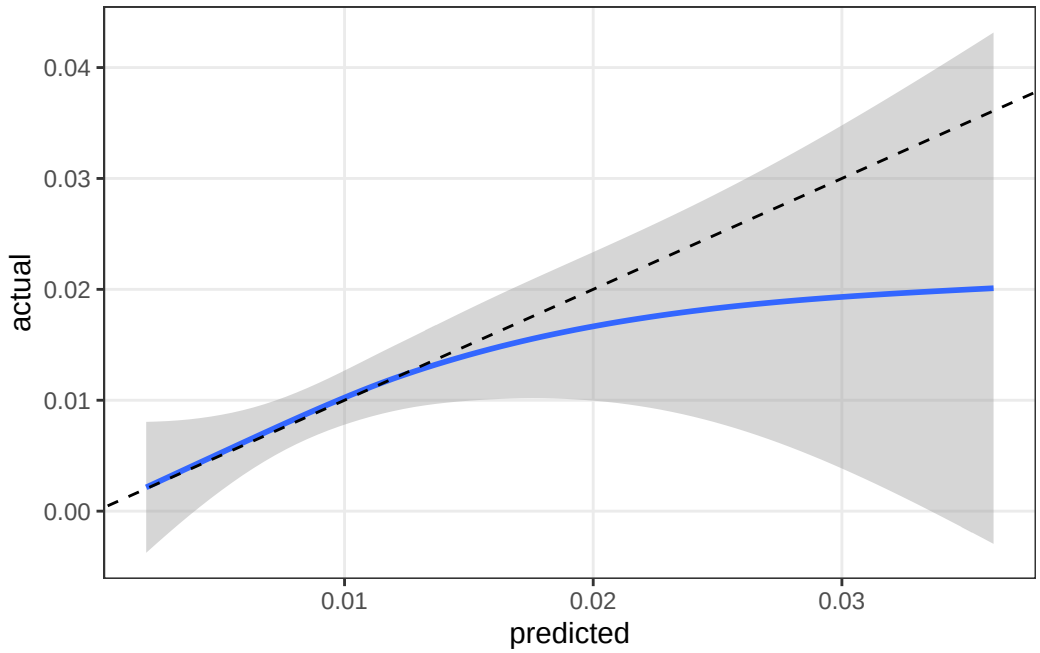
Now that we have our training model, a few cutpoints, and our simulation parameters set up, we can set up the test set and evaluate our results. This involves using `predictNMB` again to create a test set, then applying our hypothetical model and cutpoints. We'll first use our fitted model to generate some predicted probabilities. We can demonstrate the calibration using a smoother. Note that there are very few events in this dataset; the max predicted probability is only around 5%, partly because the discrimination is low (a high model discrimination will usually lead to higher predicted probabilities, and consequently a worse calibration).

```
test <- get_sample(auc, n_test, event_rate)
test$predicted_calibrated <- predict(fit, type = "response", newdata = test)

test |>
  ggplot(aes(x = predicted_calibrated, y = actual)) +
  geom_smooth() +
  geom_abline(linetype = "dashed") +
  theme_bw() +
  theme(panel.grid.minor = element_blank())
```



The model isn't actually that well calibrated! However, it's a stochastic process, so some will be OK, and some will be terrible. This is because the data used to generate the training and test sets were different. The calibration should be a bit better in the original data, at least around the value of the prevalence on the x-axis. We don't expect models to be well calibrated when we get too far above that value.

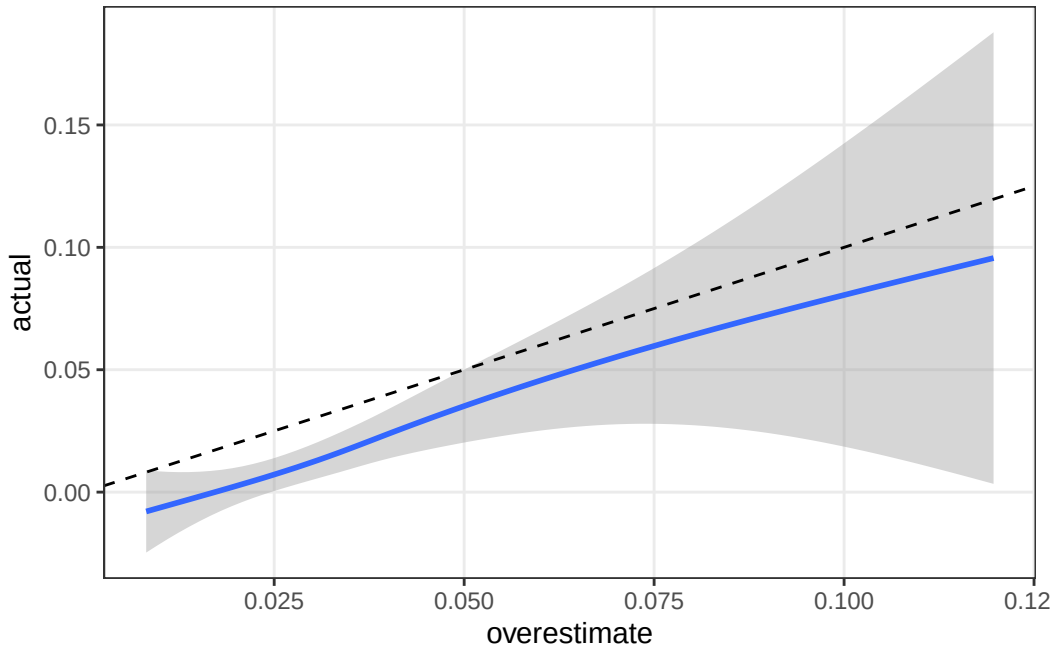
```
train |>
  ggplot(aes(x = predicted, y = actual)) +
  geom_smooth() +
  geom_abline(linetype = "dashed") +
  theme_bw() +
  theme(panel.grid.minor = element_blank())
```



Inducing miscalibration

If we were interested in the effect of miscalibration - beyond the differences we see when moving from training to testing - we can apply some systematic (monotonic) miscalibration through a power transformation. Let's say we want to induce risk overestimation in our training data using an alpha value of 3 - larger values induce more miscalibration:

```r
alpha <- 3
test$overestimate <- 1 - (1 - test$predicted_calibrated)^alpha

test |>
  ggplot(aes(x = overestimate, y = actual)) +
  geom_smooth() +
  geom_abline(linetype = "dashed") +
  theme_bw() +
  theme(panel.grid.minor = element_blank())
```



Obtaining groups by threshold

Recall that the point of this exercise was to determine the utility of the continuous prediction when prioritising patients. In a real clinical setting where there are only risk groups, no continuous estimates, what we will probably see is a combination of seeing patients based on whoever was flagged earliest and the additional information provided by clinicians. As it's not really possible to know which patients generate the most clinical concern, we can assume that a random sample will effectively mimic whoever was flagged first. In practice, this may not be too unrealistic; busy ED doctors are trying to juggle lots of patients at once, and key performance indicators usually promote getting patients admitted or discharged based on the order they come in, because anyone staying over 4 hours in some health systems gets you in trouble.

Interestingly, the NMB and NNE thresholds were high enough that only patients with a predicted risk of 0.025 or above - i.e., 2.5 times the prevalence - were included. Also interesting was that, due to our low AUC, neither patient actually experienced the event. We got a 100% false positive rate; the perils of using bad models. The Youden index was far more sensitive in that it picked up 40 patients, 2 of which ended up deteriorating. Our ranked set also picked up these two patients. We can summarise our results to also check the positive predictive value (PPV: the proportion of true positives out of all patients flagged as high risk), the sensitivity (the proportion of true positives out of everyone who went on to experience the event), and some details about the relative priority of each positive patient and their predicted risk.

```
d_subsets <- test |>
  arrange(desc(predicted_calibrated)) |>
  mutate(
    youden_pos = predicted_calibrated >= cutpoint_youden,
    nmb_pos = predicted_calibrated >= cutpoint_nmb,
    nne_pos = predicted_calibrated >= cutpoint_nne,
    rank_pos = row_number() <= n_eval
  ) |>
  pivot_longer(
    ends_with("pos"),
    names_to = "method",
    values_to = "above_threshold",
    names_transform = ~ str_remove(.x, "_pos")
  ) |>
  filter(above_threshold)

results <- d_subsets |>
  summarize(
    .by = method,
    ppv = sum(actual) / n_eval,
    sensitivity = sum(actual) / sum(test$actual),
    mean_rank = mean(which(actual == 1)),
    mean_risk = mean(predicted_calibrated)
  ) |>
  mutate(
    auc_model = auc,
    Prevalence = event_rate,
    Miscalibration = miscalibration
  )
results
```

```
# A tibble: 4 x 8
```

```
  method   ppv sensitivity mean_rank mean_risk auc_model Prevalence
  <chr>  <dbl>       <dbl>     <dbl>     <dbl>     <dbl>      <dbl>
1 youden 0.175         0.7      101.    0.0142      0.65       0.01
2 nmb    0             0         NaN    0.0336      0.65       0.01
3 nne    0             0         NaN    0.0336      0.65       0.01
4 rank   0.05          0.2         5    0.0195      0.65       0.01
# i 1 more variable: Miscalibration <chr>
```

We can see that in this iteration of the simulation, we got some fairly interesting results. The PPV for the NMB and NNE thresholds was 0 (we didn't get any true positives), and it was a not-great 0.05 for the other two. We would need to see an average of 1/0.05, or 20 patients if we randomly picked from the patients in the dataset. The sensitivity was 0.2 for the Youden and Ranking methods, indicating we missed 80% of the true positives. Most clinicians would consider this unacceptable, and this is a sign that we probably shouldn't be using this model.

However, the most interesting result comes from the mean ranking. As we note above, a PPV of 0.05 means you need to see 20 patients to get a true positive, assuming you see patients randomly. As soon as we rank patients by their predicted risk, this drops dramatically; the average rank of our positive cases - even with a poorly performing model - is 5. We can see this in the sample data.

```
which(d_subsets$actual[d_subsets$method == "youden"] == 1)
```

```
[1]   4   6  60 120 149 174 191
```

```
which(d_subsets$actual[d_subsets$method == "rank"] == 1)
```

```
[1] 4 6
```

This is a crucial result. Even when our model had a low prevalence (0.01) and a low AUC (0.65), we were still able to detect our true positives far earlier because we ranked them by their predicted risks. Randomly seeing patients (as we noted before, it's not truly random, but it's not perfect either) will mean that true positives are evenly distributed across the data. Ranking them by their predicted risk bumps them up the list.

Parallelisation and running many iterations

The easiest way to repeat the above process - 10,000 times for each combination of starting parameters in this case - is to parallelise. The setup is a little tricky at first, but the benefits quickly become apparent. This is fully detailed in the GitHub, but in brief, we want to turn

the entire process above into a function, then apply that function with different arguments depending on our start conditions. In our case, we wanted to do the resampling on both the training and test data because the training data were themselves a bit unstable, originating from a stochastic process (the sampling function from `predictNMB`). If you had a real model and real data, repeating the training step is fixed and unnecessary. See the 99_utils.R file for how to turn the above simulation into an iterative process. The setup is essentially based around `furrr::plan()`, outputting the results of each simulation as an item in a list.

```
plan(multisession, workers = availableCores())
results <- future_map(1:nrow(combs), function(i) {
  params <- combs[i, ]
  run_sims(
    event_rate = params$event_rate,
    auc = params$auc,
    miscalibration = params$miscalibration,
    niter = niter,
    n_test = n_test,
    n_eval = n_eval,
    seed = seed
  )
}, .options = furrr_options(seed = TRUE))
```

We can now do whatever analysis we want on the concatenated list, where we added a variable for the iteration number. We have already done the analysis and saved the list as an R object, so we can just load it in and take a quick peek at the summarised results.

Note that we had some results in some simulations that will break the summary (divisions by 0) so let's replace those with infinite values or 0s so they're evaluable. We can then summarise the simulations by taking the median value and interquartile ranges (IQRs) over all simulations for our estimated quantities of interest: PPV, sensitivity, mean rank and mean risk.

```
results <- bind_rows(readRDS(file = "sim_results.RDS")) |>
  mutate(
    Mean_rank = ifelse(is.nan(Mean_rank), Inf, Mean_rank),
    Mean_risk = ifelse(is.nan(Mean_risk), 0, Mean_risk)
  ) |>
  summarise(
    .by = c(Strategy, auc_model, Prevalence, Miscalibration),
    Mean_rank = median(Mean_rank),
    Mean_rank_low = quantile(Mean_rank, 0.25),
    Mean_rank_high = quantile(Mean_rank, 0.75),
    Mean_risk = median(Mean_risk),
    Mean_risk_low = quantile(Mean_risk, 0.25),
```

```
    Mean_risk_high = quantile(Mean_risk, 0.75),
    PPV_median = median(PPV),
    PPV_low = quantile(PPV, 0.25),
    PPV_high = quantile(PPV, 0.75),
    Sens_median = median(sensitivity),
    Sens_low = quantile(sensitivity, 0.25),
    Sens_high = quantile(sensitivity, 0.75)
  )
glimpse(results)
```

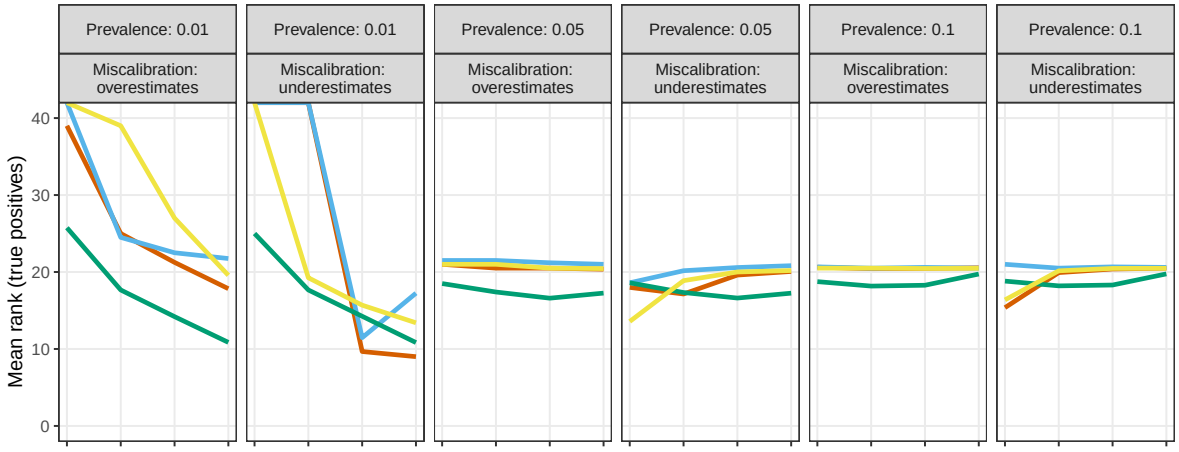
```
Rows: 144
Columns: 16
$ Strategy       <chr> "Youden", "NMB", "NNE", "Rank", "Youden", "NMB", "NNE",~
$ auc_model      <dbl> 0.65, 0.65, 0.65, 0.65, 0.65, 0.65, 0.65, 0.65, 0.65, 0~
$ Prevalence     <dbl> 0.01, 0.01, 0.01, 0.01, 0.05, 0.05, 0.05, 0.05, 0.10, 0~
$ Miscalibration <fct> none, none, none, none, none, none, none, none, none, n~
$ Mean_rank      <dbl> Inf, Inf, Inf, 25.66667, 21.00000, 20.60000, 21.00000, ~
$ Mean_rank_low  <dbl> Inf, Inf, Inf, 25.66667, 21.00000, 20.60000, 21.00000, ~
$ Mean_rank_high <dbl> Inf, Inf, Inf, 25.66667, 21.00000, 20.60000, 21.00000, ~
$ Mean_risk      <dbl> 0.01552788, 0.03118692, 0.02788169, 0.02789591, 0.07599~
$ Mean_risk_low  <dbl> 0.01552788, 0.03118692, 0.02788169, 0.02789591, 0.07599~
$ Mean_risk_high <dbl> 0.01552788, 0.03118692, 0.02788169, 0.02789591, 0.07599~
$ PPV_median     <dbl> 0.000, 0.000, 0.000, 0.025, 0.075, 0.075, 0.075, 0.125,~
$ PPV_low        <dbl> 0.000, 0.000, 0.000, 0.000, 0.050, 0.050, 0.050, 0.100,~
$ PPV_high       <dbl> 0.025, 0.025, 0.025, 0.050, 0.100, 0.125, 0.100, 0.175,~
$ Sens_median    <dbl> 0.00000000, 0.00000000, 0.00000000, 0.10000000, 0.05882~
$ Sens_low       <dbl> 0.00000000, 0.00000000, 0.00000000, 0.00000000, 0.03846~
$ Sens_high      <dbl> 0.11111111, 0.10000000, 0.06250000, 0.16666667, 0.08333~
```

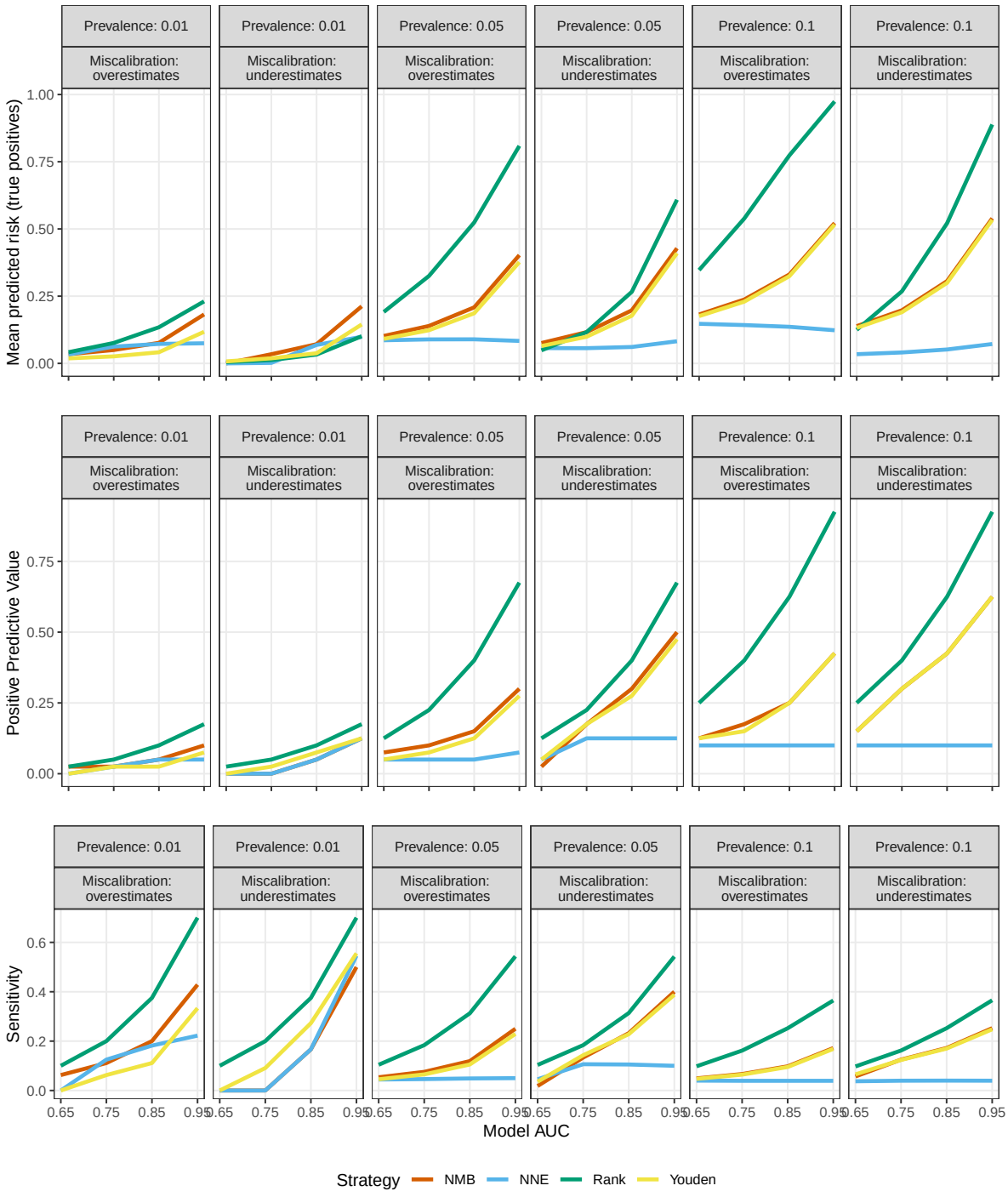
Let's look at the effect of miscalibration:

```
g_colours <- c("#D55E00", "#56B4E9", "#009E73", "#F0E442")

p <- results |>
  filter(Miscalibration != "none") |>
  mutate(Miscalibration = paste0("\n", Miscalibration)) |>
  ggplot(aes(x = auc_model))

p +
  geom_line(aes(y = Mean_rank, colour = Strategy), linewidth = 1.2) +
  facet_wrap(vars(Prevalence, Miscalibration), labeller = label_both, ncol = 6) +
```

```
  theme_bw() +
  theme(
    panel.grid.minor = element_blank(),
    axis.title.x = element_blank(),
    axis.text.x = element_blank(),
    legend.position = "none"
  ) +
  scale_x_continuous(limits = c(0.65, 0.95), breaks = seq(0.65, 0.95, 0.1)) +
  scale_y_continuous(limits = c(0, 40), breaks = seq(0, 40, 10)) +
  scale_colour_manual(values = g_colours) +
  scale_fill_manual(values = g_colours) +
  labs(y = "Mean rank (true positives)")


p +
  geom_line(aes(y = Mean_risk, colour = Strategy), linewidth = 1.2) +
  facet_wrap(vars(Prevalence, Miscalibration), labeller = label_both, ncol = 6) +
  theme_bw() +
  theme(
    panel.grid.minor = element_blank(),
    axis.title.x = element_blank(),
    axis.text.x = element_blank(),
    legend.position = "none"
  ) +
  scale_x_continuous(limits = c(0.65, 0.95), breaks = seq(0.65, 0.95, 0.1)) +
  scale_colour_manual(values = g_colours) +
  scale_fill_manual(values = g_colours) +
  labs(y = "Mean predicted risk (true positives)")


p +
  geom_line(aes(y = PPV_median, colour = Strategy), linewidth = 1.2) +
  facet_wrap(vars(Prevalence, Miscalibration), labeller = label_both, ncol = 6) +
  theme_bw() +
  theme(
    panel.grid.minor = element_blank(),
    axis.title.x = element_blank(),
    axis.text.x = element_blank(),
    legend.position = "none"
  ) +
  scale_x_continuous(limits = c(0.65, 0.95), breaks = seq(0.65, 0.95, 0.1)) +
  scale_colour_manual(values = g_colours) +
```

```
  scale_fill_manual(values = g_colours) +
  labs(y = "Positive Predictive Value")


p +
  geom_line(aes(y = Sens_median, colour = Strategy), linewidth = 1.2) +
  facet_wrap(vars(Prevalence, Miscalibration), labeller = label_both, ncol = 6) +
  theme_bw() +
  labs(
    x = "Model AUC",
    y = "Sensitivity"
  ) +
  scale_x_continuous(limits = c(0.65, 0.95), breaks = seq(0.65, 0.95, 0.1)) +
  scale_colour_manual(values = g_colours) +
  scale_fill_manual(values = g_colours) +
  theme(
    panel.grid.minor = element_blank(),
    legend.position = "bottom"
  )
```




Prevalence: 0.01
Miscalibration: overestimates
Prevalence: 0.01
Miscalibration: underestimates
Prevalence: 0.05
Miscalibration: overestimates
Prevalence: 0.05
Miscalibration: underestimates
Prevalence: 0.1
Miscalibration: overestimates
Prevalence: 0.1
Miscalibration: underestimates
Mean predicted risk (true positives)
Positive Predictive Value
Sensitivity
0.00
0.25
0.50
0.75
1.00
0.0
0.2
0.4
0.6
0.65
0.75
0.85
0.95
Model AUC
Strategy
NMB
NNE
Rank
Youden

Worked example (SERP model)

The methods from the manuscript document how we repeated a similar analytical process for the worked example. The main differences in the example were that the prevalence, AUC, and cut-off were fixed; this allowed us to examine the payoffs in the context of different capacity constraints. Where we had previously fixed the level of resource constraints at 0.4, in the example we varied these from 0.25 to 0.50 to 0.75. Likewise, our main payoff was actually an estimate of the false positive costs saved by ranking.

There were two components going on that created sampling variation: bootstrapping and Monte Carlo simulation.

Bootstrap

A bootstrap simply takes samples (in this case, without replacement) from the main dataset. We had >210,000 presentations in our dataset; this method just took repeated samples of 150 unique patients. As before, we'll just do one iteration so you can see how it works. In the GitHub, it's written up as another function - we didn't bother with parallelisation this time as it's relatively quick without it. Parallel computation can add runtime to your project if the cost of setting up all your worker CPUs is greater than the task you want them to perform.

```
# cutpoint <- 27
#
# df_sample = df_ed[sample(nrow(df_ed), 150),]
# df_positive = subset(df_sample, pred_risk >= cutpoint) |> arrange(desc(pred_risk))
#
# sample_25_rank = df_positive |> slice_head(n = floor(nrow(df_positive) * 0.25))
# sample_50_rank = df_positive |> slice_head(n = floor(nrow(df_positive) * 0.50))
# sample_75_rank = df_positive |> slice_head(n = floor(nrow(df_positive) * 0.75))
# sample_25_threshold = df_positive |> slice_sample(n = floor(nrow(df_positive) * 0.25))
# sample_50_threshold = df_positive |> slice_sample(n = floor(nrow(df_positive) * 0.50))
# sample_75_threshold = df_positive |> slice_sample(n = floor(nrow(df_positive) * 0.75))
#
# nrow(df_positive)
```

We now have 6 datasets with 3 different resource constraints (proportion evaluated) and 2 sort orders (ranked and unranked). Our resource constraint is the proportion of positive cases we can see - i.e., the proportion of cases at or above a predicted risk of 27. There were 29 positive cases, so with constraints of 0.25, 0.50 and 0.75 proportion of patients seen, we can see 7, 14 and 21 patients, give or take. The number of patients will probably change each sampling iteration based on our bootstrap.

Monte Carlo

We also wanted to determine the false positive costs associated with failing to rank. Incidentally, we can get a false negative cost too, but this was more of a secondary finding - high sensitivity is desirable (fewer false negatives), but the trade-offs required (need to see more patients) are not. We can take our inputs from the nmb sampler in the simulation but now we make them stochastic.

```
# event_rate <- mean(df_ed$outcome_died_30d) # Rate of 30d mortality observed in the data
#
# deterioration_cost = rnorm(10000, 14345, 696) # see Curtis et al
#
# FP = rgamma(10000, shape = 110.314, scale = 0.172) * 1.50 + # cost of clinical time - see l
#       (event_rate * (1 - rnorm(10000, 0.910, 0.036)) * deterioration_cost) # opportunity co
# FN = deterioration_cost
#
# hist(FP, main = "False positive costs (N = 10,000)", xlab = "Cost in 2025 SGD")
```

Putting it all together

Over lots of iterations (10,000 bootstrap iterations and 10,000 Monte Carlo simulations) we can get a similar set of graphics as before. We combine our results using the first iteration of the false positive and false negative costs (normally we'd only generate one sample per iteration):

```
# results <- tibble(
#       Strategy = c(rep("Ranked", 3), rep("Unranked", 3)),
#       Pct_evaluated = rep(c(25, 50, 75), 2),
#       PPV = c(
#         sum(sample_25_rank$outcome_died_30d)/nrow(sample_25_rank),
#         sum(sample_50_rank$outcome_died_30d)/nrow(sample_50_rank),
#         sum(sample_75_rank$outcome_died_30d)/nrow(sample_75_rank),
#         sum(sample_25_threshold$outcome_died_30d)/nrow(sample_25_threshold),
#         sum(sample_50_threshold$outcome_died_30d)/nrow(sample_50_threshold),
#         sum(sample_75_threshold$outcome_died_30d)/nrow(sample_75_threshold)
#       ),
#       Sensitivity = c(
#         sum(sample_25_rank$outcome_died_30d)/sum(df_sample$outcome_died_30d),
#         sum(sample_50_rank$outcome_died_30d)/sum(df_sample$outcome_died_30d),
#         sum(sample_75_rank$outcome_died_30d)/sum(df_sample$outcome_died_30d),
#         sum(sample_25_threshold$outcome_died_30d)/sum(df_sample$outcome_died_30d),
```

```
#         sum(sample_50_threshold$outcome_died_30d)/sum(df_sample$outcome_died_30d),
#         sum(sample_75_threshold$outcome_died_30d)/sum(df_sample$outcome_died_30d)
#       ),
#       FP_cost = c(
#         sum(sample_25_rank$outcome_died_30d == 0) * FP[1],
#         sum(sample_50_rank$outcome_died_30d == 0) * FP[1],
#         sum(sample_75_rank$outcome_died_30d == 0) * FP[1],
#         sum(sample_25_threshold$outcome_died_30d == 0) * FP[1],
#         sum(sample_50_threshold$outcome_died_30d == 0) * FP[1],
#         sum(sample_75_threshold$outcome_died_30d == 0) * FP[1]
#       ),
#       Misclassification_cost = c(
#         sum(sample_25_rank$outcome_died_30d == 0) * FP[1] +
#           (sum(df_sample$outcome_died_30d == 1) - sum(sample_25_rank$outcome_died_30d == 1)
#         sum(sample_50_rank$outcome_died_30d == 0) * FP[1] +
#           (sum(df_sample$outcome_died_30d == 1) - sum(sample_50_rank$outcome_died_30d == 1)
#         sum(sample_75_rank$outcome_died_30d == 0) * FP[1] +
#           (sum(df_sample$outcome_died_30d == 1) - sum(sample_75_rank$outcome_died_30d == 1)
#         sum(sample_25_threshold$outcome_died_30d == 0) * FP[1] +
#           (sum(df_sample$outcome_died_30d == 1) - sum(sample_25_threshold$outcome_died_30d
#         sum(sample_50_threshold$outcome_died_30d == 0) * FP[1] +
#           (sum(df_sample$outcome_died_30d == 1) - sum(sample_50_threshold$outcome_died_30d
#         sum(sample_75_threshold$outcome_died_30d == 0) * FP[1] +
#           (sum(df_sample$outcome_died_30d == 1) - sum(sample_75_threshold$outcome_died_30d
#       )
# )
# results
```

We now see that ranking can save costs and improve model performance, as with the simulation. This benefit increases as the resources get more constrained. While our primary interest wasn't in false negatives, we can see that a missed deterioration event can be very expensive; this is a great incentive to use ranking if your model has a good AUC. Repeating this process is fairly simple - see the GitHub.

If you have any questions, feel free to email the corresponding author.